

ui.code 全面详解

`ui.code` 是 NiceGUI 基于 Prism.js 封装的代码展示与编辑组件，核心用于网页端语法高亮显示代码片段，支持代码编辑、复制、语言切换、主题切换等功能，适用于技术文档、代码演示、在线调试工具等场景。以下从核心特性、配置参数、使用方法、进阶用法等维度展开全面解析。

一、核心定位与基础特性

1. 核心功能

- 语法高亮：支持 200+ 编程语言（如 Python、JavaScript、HTML、CSS 等），基于 Prism.js 实现精准语法着色。
- 代码编辑：可配置为只读或可编辑模式，编辑模式支持自动缩进、换行等基础编辑功能。
- 交互操作：内置复制按钮（一键复制代码）、语言标识显示、行号显示（可关闭）。
- 样式定制：支持多个预设主题（如浅色、深色、编程风格主题），可自定义字体大小、行高、容器样式。
- 数据绑定：支持与其他组件双向绑定，实时同步代码内容。
- 高级特性：支持代码折叠、自定义语法规则、行高亮等进阶功能（通过 Prism.js 插件扩展）。

2. 基础用法

通过 `ui.code()` 传入代码字符串和语言类型，快速创建语法高亮的代码块，示例代码结构如下：

```
from nicegui import ui

# 基础用法：只读模式，Python 语法高亮
python_code = '''def hello_nicegui():
    print("Hello, NiceGUI!")
    return "ui.code 语法高亮示例"'''

ui.code(python_code, language='python').classes('w-full')

ui.run()
```

二、关键配置参数

`ui.code` 的初始化参数涵盖代码内容、语法规则、交互行为、样式配置等维度，以下是完整参数说明：

参数名	类型	说明	默认值	版本特性
value	字符串	代码内容（支持换行符 <code>\n</code> 、制表符 <code>\t</code> ）	""	-
language	字符串	编程语言标识（需符合 Prism.js 语言名称，如 'python'、'javascript'、'html'）	"text"	-
readonly	布尔值	是否为只读模式（False 为可编辑模式）	True	-

参数名	类型	说明	默认值	版本特性
line_numbers	布尔值	是否显示行号	True	-
copyable	布尔值	是否显示复制按钮	True	-
theme	字符串	语法高亮主题（支持预设主题，见下文「主题列表」）	"default"	-
font_size	数值 / 字符串	字体大小（支持 CSS 单位，如 14、'14px'、'1.2rem'）	14	-
line_height	数值 / 字符串	行高（支持 CSS 单位，如 1.5、'1.5'、'18px'）	1.5	-
wrap	布尔值	是否自动换行（False 为横向滚动）	False	-
on_change	回调函数	可编辑模式下内容变更时触发，参数为 <code>ValueChangeEventArguments</code>	-	-
placeholder	字符串	可编辑模式下无内容时的占位提示	""	2.17.0 版本新增
html_id	字符串	组件 DOM 元素的 ID，用于自定义 CSS/JS 操作	None	2.16.0 版本新增
additional_resources	列表	额外加载的 Prism.js 插件资源（CSS/JS URL），用于扩展功能	None	2.11.0 版本新增

核心补充说明

1. 支持的编程语言（部分常用）

语言	Prism 标识	示例
Python	'python'	<code>ui.code(code, language='python')</code>
JavaScript	'javascript'	<code>ui.code(code, language='javascript')</code>
HTML	'html'	<code>ui.code(code, language='html')</code>
CSS	'css'	<code>ui.code(code, language='css')</code>
JSON	'json'	<code>ui.code(code, language='json')</code>
SQL	'sql'	<code>ui.code(code, language='sql')</code>
Markdown	'markdown'	<code>ui.code(code, language='markdown')</code>

语言	Prism 标识	示例
无语法高亮	'text'	<code>ui.code(code, language='text')</code>

完整语言列表参考 [Prism.js 官方文档](#)。

2. 预设主题列表

主题名称	说明	适用场景
"default"	默认浅色主题	通用文档、浅色背景页面
"dark"	深色主题	夜间模式、深色背景页面
"coy"	编程风格浅色主题	技术博客、教程文档
"solarized-light"	日光色浅色主题	长时间阅读场景
"solarized-dark"	日光色深色主题	夜间开发、低亮度环境
"tomorrow"	柔和浅色主题	通用场景，视觉友好
"tomorrow-night"	柔和深色主题	夜间使用，不刺眼

三、核心功能用法

1. 基础展示功能

(1) 只读模式（默认）

用于展示代码片段，支持复制、行号显示，示例：

```
from nicegui import ui

# 展示 HTML 代码，关闭行号，使用深色主题
html_code = '''<div class="container">
  <h1>NiceGUI ui.code 示例</h1>
  <p>语法高亮支持 HTML/CSS/JS</p>
</div>'''

ui.code(
    html_code,
    language='html',
    line_numbers=False,
    theme='dark',
    font_size=12
).classes('w-full h-48')

ui.run()
```

(2) 可编辑模式

设置 `readonly=False` 启用编辑功能，支持内容变更监听，示例：

```
from nicegui import ui
from nicegui.events import ValueChangeEventArguments

def on_code_change(e: ValueChangeEventArguments):
    # e.value 为当前编辑的代码内容
    code_length = len(e.value) if e.value else 0
    ui.notify(f'代码长度: {code_length} 字符')

# 可编辑的 Python 代码块，添加占位提示
editor = ui.code(
    value='',
    language='python',
    readonly=False,
    placeholder='请输入 Python 代码...',
    on_change=on_code_change
).classes('w-full h-64')

# 按钮控制代码清空
ui.button('清空代码', on_click=lambda: editor.set_value(''))

ui.run()
```

2. 样式定制

(1) 字体与行高配置

通过 `font_size` 和 `line_height` 调整代码显示样式，支持 CSS 单位：

```
from nicegui import ui

code = '''# 自定义字体大小和行高
def calculate(a: int, b: int) -> int:
    return a + b'''

ui.code(
    code,
    language='python',
    font_size='16px', # 字体大小 16px
    line_height=1.8, # 行高 1.8 倍
    theme='coy'
).classes('w-full')

ui.run()
```

(2) 自动换行与容器样式

设置 `wrap=True` 启用自动换行，结合 Tailwind CSS 类修改容器样式（如边框、圆角）：

```
from nicegui import ui

# 长代码片段，启用自动换行
long_code = '''def long_function_name(parameter1: str, parameter2: int, parameter3: list,
parameter4: dict) -> bool:
    """这是一个长函数注释，用于说明函数的功能、参数和返回值，启用自动换行后不会出现横向滚动条"""
    if parameter1 and parameter2 > 0 and len(parameter3) > 0 and 'key' in parameter4:
        return True
    return False'''

ui.code(
    long_code,
    language='python',
    wrap=True,
    line_numbers=True
).classes('w-full h-48 border-2 border-gray-300 rounded-lg p-4')

ui.run()
```

3. 主题切换

通过动态修改 `theme` 属性，实现主题实时切换：

```
from nicegui import ui

code = '''print("主题切换示例")
for i in range(5):
    print(f"当前索引: {i}")'''

editor = ui.code(
    code,
    language='python',
    theme='default'
).classes('w-full h-48')

# 主题切换按钮组
with ui.row().classes('mt-4'):
    ui.button('默认主题', on_click=lambda: editor.set_theme('default'))
    ui.button('深色主题', on_click=lambda: editor.set_theme('dark'))
    ui.button('Solarized 深色', on_click=lambda: editor.set_theme('solarized-dark'))
    ui.button('Coy 主题', on_click=lambda: editor.set_theme('coy'))

ui.run()
```

4. 数据绑定

与其他组件（如 `ui.input`、`ui.button`）双向绑定，实现代码内容同步：

```
from nicegui import ui

# 初始代码
initial_code = 'print("Hello, Binding!")'

# 代码编辑器
code_editor = ui.code(
    initial_code,
    language='python',
    readonly=False
).classes('w-full h-48')

# 输入框绑定代码内容（实时同步）
ui.input('修改代码内容', value=initial_code).bind_value(code_editor, 'value')

# 按钮显示当前代码
ui.button('显示代码', on_click=lambda: ui.notify(f'当前代码: \n{code_editor.value}'))

ui.run()
```

四、进阶用法

1. 扩展 Prism.js 插件（如代码折叠）

通过 `additional_resources` 加载 Prism.js 插件，扩展核心功能（如代码折叠、行高亮）。以下示例加载「代码折叠」插件：

```
from nicegui import ui

# 加载 Prism.js 代码折叠插件（CSS + JS）
code = '''def foldable_function():
    # 折叠区域示例
    print("这部分代码可以折叠")
    for i in range(3):
        print(i)

def another_function():
    print("另一部分代码")'''

ui.code(
    code,
    language='python',
    additional_resources=[
        # 折叠插件 CSS
        'https://cdn.jsdelivr.net/npm/prismjs@1.29.0/plugins/折叠/prism-fold.min.css',
        # 折叠插件 JS
        'https://cdn.jsdelivr.net/npm/prismjs@1.29.0/plugins/折叠/prism-fold.min.js',
```

```
]
).classes('w-full h-64')

ui.run()
```

2. 代码导出功能

结合 `ui.download` 组件，将代码内容导出为文件（如 `.py`、`.js`）：

```
from nicegui import ui

code = '''def hello_nicegui():
    print("Hello, NiceGUI!")
    return "代码导出示例"'''

code_editor = ui.code(
    code,
    language='python',
    readonly=False
).classes('w-full h-48')

# 导出代码为 .py 文件
def export_code():
    content = code_editor.value or '# 无代码内容'
    file_name = f'code_export.{code_editor.language}' # 文件名后缀与语言匹配
    ui.download(content, file_name, f'text/{code_editor.language}')

ui.button('导出代码文件', on_click=export_code).classes('mt-4 bg-green-500 text-white')

ui.run()
```

3. 代码格式化（结合第三方库）

使用 `black`（Python 代码格式化库）实现代码自动格式化，示例：

```
from nicegui import ui
import black
from black import Mode

def format_python_code():
    try:
        # 使用 black 格式化代码
        formatted_code = black.format_str(
            code_editor.value,
            mode=Mode(line_length=88) # 行宽限制 88
        )
        code_editor.set_value(formatted_code)
        ui.notify('代码格式化成功!', type='success')
    except Exception as e:
        ui.notify(f'格式化失败: {str(e)}', type='error')

# 可编辑的 Python 代码块
```

```
code_editor = ui.code(
    value='def unformatted_function(a,b,c):\nreturn a+b+c',
    language='python',
    readonly=False
).classes('w-full h-64')

ui.button('格式化代码', on_click=format_python_code).classes('mt-4 bg-blue-500 text-white')

# 安装依赖: pip install black
ui.run()
```

4. 自定义复制行为

通过 `add_slot` 自定义复制按钮的功能（如复制时添加版权注释）：

```
from nicegui import ui

code = '''def custom_copy_example():
    print("复制时会添加版权注释")'''

code_editor = ui.code(
    code,
    language='python',
    copyable=False # 隐藏默认复制按钮
).classes('w-full h-48')

# 自定义复制按钮：复制代码并添加版权注释
code_editor.add_slot('after', '''
    <q-btn
        label="复制代码（带版权）"
        size="sm"
        icon="content_copy"
        @click="copyWithCopyright"
        class="mt-2"
    />
    <script>
        function copyWithCopyright() {
            const code = this.$parent.value;
            const copyright = `\\n\\n# 版权所有 © 2024 NiceGUI 示例`;
            navigator.clipboard.writeText(code + copyright).then(() => {
                this.$q.notify({ message: '复制成功（已添加版权）', type: 'success' });
            });
        }
        this.copyWithCopyright = copyWithCopyright.bind(this);
    </script>
    ''')

ui.run()
```


5. 行高亮功能（通过 Prism 插件）

加载「行高亮」插件，实现指定行的高亮显示（如标记关键代码行）：

```
from nicegui import ui

# 加载行高亮插件
code = '''def highlight_example():
    print("这是普通行")
    print("这是关键行（会高亮）") # 高亮第 2 行
    print("这也是关键行（会高亮）") # 高亮第 3 行
    print("这是普通行")'''

ui.code(
    code,
    language='python',
    additional_resources=[
        'https://cdn.jsdelivr.net/npm/prismjs@1.29.0/plugins/line-highlight/prism-line-highlight.min.css',
        'https://cdn.jsdelivr.net/npm/prismjs@1.29.0/plugins/line-highlight/prism-line-highlight.min.js',
    ],
    # 通过 props 传递行高亮参数（高亮第 2-3 行）
    props={'data-line': '2-3'}
).classes('w-full h-64')

ui.run()
```

五、核心方法速查表

方法	说明	参数	示例
set_value(value)	设置代码内容	value：字符串（支持换行、制表符）	<code>editor.set_value('print("新代码")')</code>
get_value()	获取当前代码内容	-	<code>content = editor.get_value()</code>
set_language(language)	动态修改编程语言	language：Prism 语言标识（如 'javascript'）	<code>editor.set_language('javascript')</code>
set_theme(theme)	动态修改高亮主题	theme：预设主题名称（如 'dark'）	<code>editor.set_theme('solarized-dark')</code>
set_readonly(readonly)	切换只读 / 可编辑模式	readonly：布尔值（True/False）	<code>editor.set_readonly(False)</code>
update()	刷新组件显示	-	动态修改属性后调用，确保界面同步
on_value_change(callback)	绑定内容变更回调	callback：回调函数（参数为 <code>ValueChangeEventArguments</code> ）	<code>editor.on_value_change(lambda e: print(e.value))</code>
add_slot(name, template)	添加 Vue 插槽（自定义 UI 元素）	name：插槽名称（如 'after'）；template：Vue 模板字符串	见「自定义复制行为」示例

六、注意事项与最佳实践

1. 插件兼容性

- 加载 Prism.js 插件时，需确保插件版本与 Prism.js 核心库兼容（NiceGUI 内置 Prism.js 1.29.0 版本）。
- 插件资源 URL 建议使用 jsdelivr 等 CDN，确保加载速度和稳定性。

2. 性能优化

- 大数据量代码（超过 1000 行）：建议关闭行号（`line_numbers=False`）和自动换行（`wrap=False`），避免 DOM 元素过多导致卡顿。
- 可编辑模式：避免频繁触发 `on_change` 回调（如每输入一个字符触发），可通过防抖函数优化：

```
from nicegui import ui
from functools import lru_cache, wraps
import time

# 防抖装饰器
def debounce(seconds=0.5):
    def decorator(func):
        last_call = 0
        @wraps(func)
        def wrapper(*args, **kwargs):
            nonlocal last_call
            now = time.time()
            if now - last_call > seconds:
                last_call = now
                return func(*args, **kwargs)
        return wrapper
    return decorator

# 防抖回调（0.5 秒内仅触发一次）
@debounce(0.5)
def on_code_change(e):
    ui.notify(f'代码已更新（防抖）')

editor = ui.code(language='python', readonly=False,
on_change=on_code_change).classes('w-full h-64')
ui.run()
```

3. 内容安全

- 可编辑模式下，若代码用于后端执行（如在线代码运行工具），需严格过滤危险操作（如文件读写、系统命令执行），避免安全漏洞。
- 避免在代码中展示敏感信息（如密码、Token、API 密钥）。

4. 样式定制注意事项

- 自定义 `font_size` 和 `line_height` 时，建议使用相对单位（如 `rem`），适配不同屏幕尺寸。
- 结合 Tailwind CSS 类修改容器样式时，避免覆盖组件内置样式（如 `p-0` 会清除默认内边距）。

总结

`ui.code` 是 NiceGUI 中功能强大的代码展示与编辑组件，通过封装 Prism.js 实现了高效的语法高亮和丰富的交互功能。其核心优势在于：

1. 简洁的 Python API 与前端语法高亮能力的无缝结合，无需手动配置 Prism.js；
2. 支持只读 / 可编辑模式切换，适配文档展示和在线编辑场景；
3. 丰富的样式定制和插件扩展能力，满足个性化需求；
4. 与 NiceGUI 生态深度集成，支持数据绑定、组件联动等功能。

适用于技术文档、代码演示平台、在线调试工具、后台管理系统的代码配置模块等场景。对于复杂需求（如多人协作编辑、实时运行代码），可结合第三方库（如 `black` 格式化、`exec` 代码执行）进一步扩展。